

A COURSE COMPANION



NumPy Book

Lessons & Explanations

A lesson-by-lesson walkthrough of **NumPy**, built from the YouTube course by **JoeTech**.

What's inside

01

Introduction to NumPy

3

JoeTech, NumPy Course, Video 1 — Introduction to NumPy

Introduction to NumPy

JoeTech, NumPy Course, Video 1 — Introduction to NumPy

Every data-heavy Python program eventually runs into the same wall: plain Python is not built for crunching large amounts of numbers. **NumPy** — short for *Numerical Python* — is the open-source, third-party library that removes that wall. It gives Python a fast, memory-efficient way to store and operate on large, multi-dimensional collections of numbers, called **arrays** and **matrices**, and ships with a huge collection of mathematical functions built to work directly on them.

It isn't a small niche tool, either. NumPy sits at the foundation of almost the entire Python data and scientific-computing stack — pandas, scikit-learn, TensorFlow, PyTorch, and dozens of others are all built on top of the NumPy array. Learning it properly now is what makes every one of those tools make sense later. The project itself is fully open source, and its code lives publicly on GitHub at github.com/numpy/numpy.

Why use a NumPy array instead of a Python list?

You *can* store numbers in a plain Python list — so why bother with a whole separate library? Because a Python list was designed to hold anything (numbers, strings, other lists, mixed together), and that generality has a cost. NumPy's array gives up that generality on purpose, and gets several concrete advantages back in return:

ADVANTAGE	WHAT IT MEANS IN PRACTICE
Consumes less memory	Elements are packed tightly, with no per-object overhead
Much faster	Bulk operations run in optimized, compiled C — not a Python <code>for</code> loop
Easy to use	One array handles what would otherwise take nested loops and helper functions
Element-wise operations	Apply an operation to every element at once — no explicit loop needed

ADVANTAGE

WHAT IT MEANS IN PRACTICE

Contiguous storage

Elements sit back-to-back in memory, which is *why* the first three points are true

That last row is the key that unlocks the other three. A Python list is really a list of *pointers* — each element can live anywhere in memory, and Python has to chase each pointer separately to read a value. A NumPy array stores its actual numbers in one unbroken block of memory instead. That single design choice is what lets NumPy hand off bulk math to fast, compiled routines (and take advantage of CPU-level tricks like SIMD) instead of interpreting a loop one element at a time in Python.

```
import numpy as np
import time

n = 2_000_000
python_list = list(range(n))
numpy_array = np.arange(n)

start = time.time()
squared_list = [x * x for x in python_list]
print("Python list:", time.time() - start, "sec")

start = time.time()
squared_array = numpy_array ** 2 # element-wise: no explicit loop
print("NumPy array:", time.time() - start, "sec")
```

Run this yourself and the NumPy version will typically finish several times faster — and the gap only grows as `n` gets larger.

Homogeneous vs. heterogeneous data

These two words come up constantly once you start reading about NumPy, so it's worth nailing them down early:

- **Heterogeneous** — a collection that can hold *different* types of objects together.
- **Homogeneous** — a collection where every item is the *same* type.

A plain Python list is heterogeneous by default — nothing stops you from mixing types in the same list:

```
mixed = [1, "two", 3.0, [4]]  # perfectly legal Python list
```

A NumPy array does not allow this. **Every item in a NumPy array has to be of the same type.** If you hand it mixed types, NumPy will upcast everything to one common type rather than raising an error:

```
arr = np.array([1, "two", 3.0])
print(arr)      # ['1' 'two' '3.0'] -- everything became a string
print(arr.dtype) # <U32
```

WATCH OUT

Mixing types into `np.array()` won't crash your program — it will silently upcast every element to whatever type can represent them all (often turning numbers into strings, as above). This is rarely what you want, so keep your input data one consistent type.

This homogeneity requirement is not a limitation for its own sake — it's what makes the rest of the story work. Because every element is guaranteed to be the same type, NumPy knows *exactly* how many bytes each element needs, and therefore exactly how much storage the whole array requires, before it allocates a single byte. That predictability is what allows the tightly packed, contiguous memory layout from the previous section.

Like Python lists, NumPy arrays are also **zero-indexed** — the first element is at position `0`, not `1`.

Getting started

NumPy is a third-party package, so it needs to be installed once per environment before you can import it:

```
pip install numpy
```

By convention, almost every piece of NumPy code you will ever read imports the library under the short alias `np`:

```
import numpy as np

print(np.__version__)  # confirms the install and shows the version, e.g.
1.26.4
```

If that line runs without an error, you're set up correctly for the rest of the course.

KEY TAKEAWAYS

- NumPy ("Numerical Python") is an open-source library for fast, memory-efficient arrays and matrices, and it underpins most of Python's data/scientific ecosystem.
- NumPy arrays beat Python lists on memory, speed, and ease of use — mainly because their elements are stored in one **contiguous** block of memory instead of scattered pointers.
- A Python list is **heterogeneous** by default; a NumPy array is **homogeneous** — every element must share one `dtype`.
- Because the type is fixed, NumPy always knows the exact storage size an array needs in advance.
- Arrays are zero-indexed, install with `pip install numpy`, and are conventionally imported as `import numpy as np`.